DAY 8

File I/O

Reading, Writing, CSV & JSON

PHASE 3 · INTERMEDIATE

2 hrs	5	3	20
Session Length	Core Topics	Concept Files	Practice Problems

30-Day Python Programming Course
Instructor Session Guide

Session Overview

Goal: Give students the ability to persist data beyond a single program run — reading and writing plain text files, tabular CSV data, and structured JSON data, the three formats they'll use constantly in real-world projects (including the course capstone).

Outcome: By the end of this session, students can open files safely with `with`, read and write text line by line, export/import tabular data with the `csv` module, and serialize/deserialize structured data with the `json` module.

Timeline

Time	Segment	Focus
0:00 – 0:20	File Basics	open/close, with statement, read/write/append
0:20 – 0:35	Reading Patterns	readline, readlines, iterating line by line
0:35 – 0:40	Break	Short breather
0:40 – 1:10	csv Module	csv.writer/reader, DictWriter/DictReader
1:10 – 1:40	json Module	dump/load, dumps/loads, nested JSON
1:40 – 1:55	Guided Practice	Students work Section A-C of problem set
1:55 – 2:00	Wrap-Up	Recap + preview Day 9 (Error Handling)

> **TIP**
>
> File I/O is where Python code stops being purely academic — every real app reads configs, logs, exports reports, or talks to an API using these exact patterns. Emphasize the **with** statement early; it prevents an entire category of bugs students will otherwise hit for years.

Prerequisites & Preparation

Requirements

Requirement	Details
Completed	Day 7 — Strings & String Methods
Python version	3.12 (installed on Day 1)
Editor	VS Code with Python + Pylance extensions
New modules	csv, json (both built into the standard library — no install needed)

Instructor Pre-Session Checklist

- ■ Confirm every student can create/write/read a file in their working folder (no permissions issues)
- ■ Pre-create an empty 'day08_data' scratch folder as a demo convention
- ■ Have a sample CSV and JSON file ready to open in VS Code's built-in viewer
- ■ Remind students: never use real input() in demo files — pre-assigned variables only
- ■ Test that today's three concept files run cleanly end-to-end before the session

WATCH OUT

Windows students sometimes see extra blank lines in CSV output — this is why every CSV example opens with `newline=""`. Mention this explicitly to save debugging time later.

Step-by-Step Instruction Guide

STEP 1 — Opening, Closing & the with Statement

"Every program we've written so far forgets everything the moment it ends. Today that changes — we're going to read and write real files on disk, starting with plain text."

```
# Manual open/close — risky if an error happens in between
f = open("diary.txt", "w")
f.write("Day 1: Arrived in Mumbai for the new job.\n")
f.close()

# Preferred: with statement auto-closes the file
with open("diary.txt", "a") as f:
    f.write("Day 3: Tried vada pav near the office.\n")
```

Common Errors

Error	Cause	Fix
FileNotFoundError	Reading a file with mode 'r' that doesn't exist	Use 'w' to create it, or check os.path.exists() first
Empty file after write	Forgot to call close(), or wrote outside the with block	Always use the with statement
Old content survives	Opened with 'a' instead of 'w' when a fresh file was wanted	Use 'w' to overwrite, 'a' to append

DONE WHEN

Done When: every student has created a file, appended to it with the with statement, and printed its contents back with read().

STEP 2 — Reading Patterns: read(), readline(), readlines()

"There's more than one way to read a file. read() grabs everything at once — fine for small files, wasteful for huge ones. Looping over the file object directly is the most memory-efficient way."

```
with open("diary.txt", "r") as f:
    for line_number, line in enumerate(f, start=1):
        print(f"Line {line_number}: {line.strip()}")

Line 1: Day 1: Arrived in Mumbai for the new job.
Line 2: Day 2: Explored Marine Drive in the evening.
```

> **TIP**
> readlines() returns a list where every string still has its trailing '\n' — always .strip() before printing or comparing.

STEP 3 — The csv Module

"Almost every spreadsheet, bank statement, and data export you'll ever touch is a CSV file underneath. Python's csv module handles the fiddly bits — like commas inside values — that would break naive string splitting."

```
import csv

fieldnames = ["name", "city", "salary"]
with open("employees.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerow({"name": "Priya Sharma", "city": "Mumbai", "salary": 65000})

with open("employees.csv", "r", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["city"])

Priya Sharma Mumbai
```

WATCH OUT

Every value that comes back from csv.reader / DictReader is a string — even numbers. Always int() or float() a numeric column before doing math on it.

STEP 4 — The json Module

"JSON is the language APIs speak — Swiggy, IRCTC, Flipkart, all of them. The great news is JSON maps almost one-to-one onto Python dicts and lists, so this will feel very familiar."

```
import json

customer = {"name": "Priya Sharma", "city": "Mumbai", "orders": 4}
with open("customer.json", "w") as f:
    json.dump(customer, f, indent=4)

with open("customer.json", "r") as f:
    loaded = json.load(f)
print(loaded["name"])

Priya Sharma
```

DONE WHEN

Done When: students have written a nested dict (dict containing a list of dicts) to JSON and read a specific nested value back out.

Session Wrap-Up

Student Checklist

- ■ I can open, write to, and safely close a file using the with statement
- ■ I know the difference between 'r', 'w', and 'a' modes
- ■ I can read a file with read(), readline(), readlines(), or by iterating directly
- ■ I can write and read tabular data with csv.writer/reader and DictWriter/DictReader
- ■ I can serialize a Python dict/list to JSON and load it back with json.dump/load

Key Concepts Recap

File I/O is really three related skills stacked together: (1) the low-level open/read/write/close mechanics with the with statement as the safe default, (2) the csv module for anything row-and-column shaped, and (3) the json module for anything nested or API-shaped. Nearly every real project — including the course capstone — touches all three.

Next Day Preview

Day 9 — Error Handling: try/except/else/finally, raising exceptions manually, building custom exception classes, and logging basics. Students will learn to handle exactly the kinds of failures file I/O can cause (missing files, bad permissions, malformed data).

> **TIP**
>
> If students run short on time, cut Step 2 (reading patterns) short — the with-statement and csv/json sections are higher priority for the capstone project later in the course.

Quick Reference Card

File Modes

Mode	Meaning
'r'	Read (default). File must already exist.
'w'	Write. Creates file if missing, ERASES existing content.
'a'	Append. Creates file if missing, adds to the end.
'r+'	Read and write without erasing existing content.

Daily Workflow Commands

```
python day08_01_file_basics.py
python day08_02_csv_module.py
python day08_03_json_module.py
python day08_practise_problems_01_file_basics.py
```

VS Code Shortcuts

Shortcut (Win/Linux)	Shortcut (Mac)	Action
Ctrl+`	Cmd+`	Toggle integrated terminal
F5	F5	Run/Debug current file
Ctrl+Shift+P	Cmd+Shift+P	Open command palette

Tool Summary

Tool / Module	Use For
open() / with	Low-level text file reading and writing
csv module	Tabular, spreadsheet-style data
json module	Structured/nested data, API payloads, configs